LOCAL LLM & CUSTOMIZED HARNESS · CODE WALKTHROUGH

The Main Harness: Components & Relations

How the files in Harness/harness/ fit together and talk to each other

Components

Relations

Call flows

Extension points

Companion to Session 2 · Modules M1–M7 + M14

BIG PICTURE

One package, three jobs

Talk to the model

Harness facade wraps the connector, config, prompts and logger. One call: h.llm(profile, messages).

- `harness.py`
- `llm_client.py  (M1)`
- `config.py + config.yaml  (M2)`
- `prompts.py + prompts/  (M3)`
- `logger.py  (M14)`

Trust the output

Constrain the shape, then validate it twice and recover when it fails.

- `schemas.py  (M4)`
- `validation.py  (M5)`
- `data_store.py  (domain data)`

Act safely

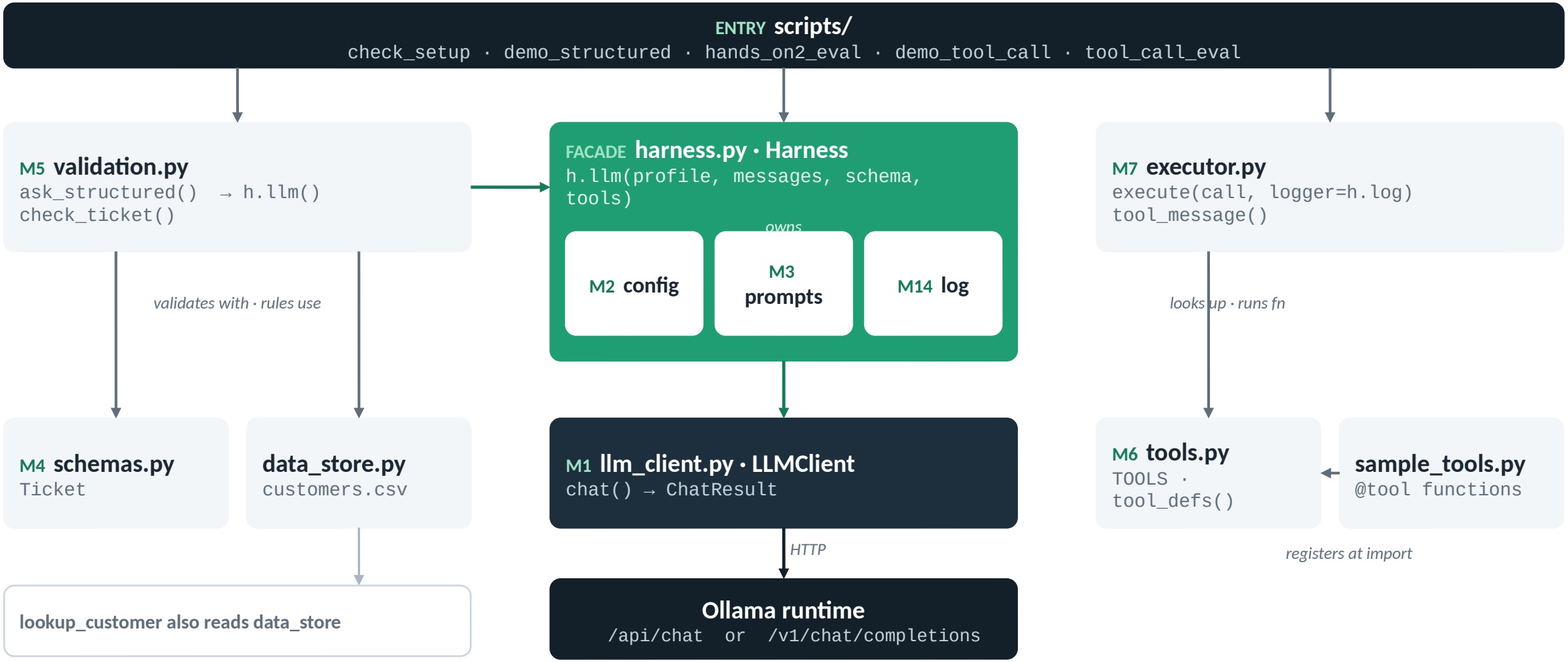
Tools are declared and typed; the executor runs only calls that pass every check.

- `tools.py  (M6)`
- `executor.py  (M7)`
- `sample_tools.py`

scripts/ are thin entry points: they build messages, call these components, and print or save results.

ARCHITECTURE

Component map



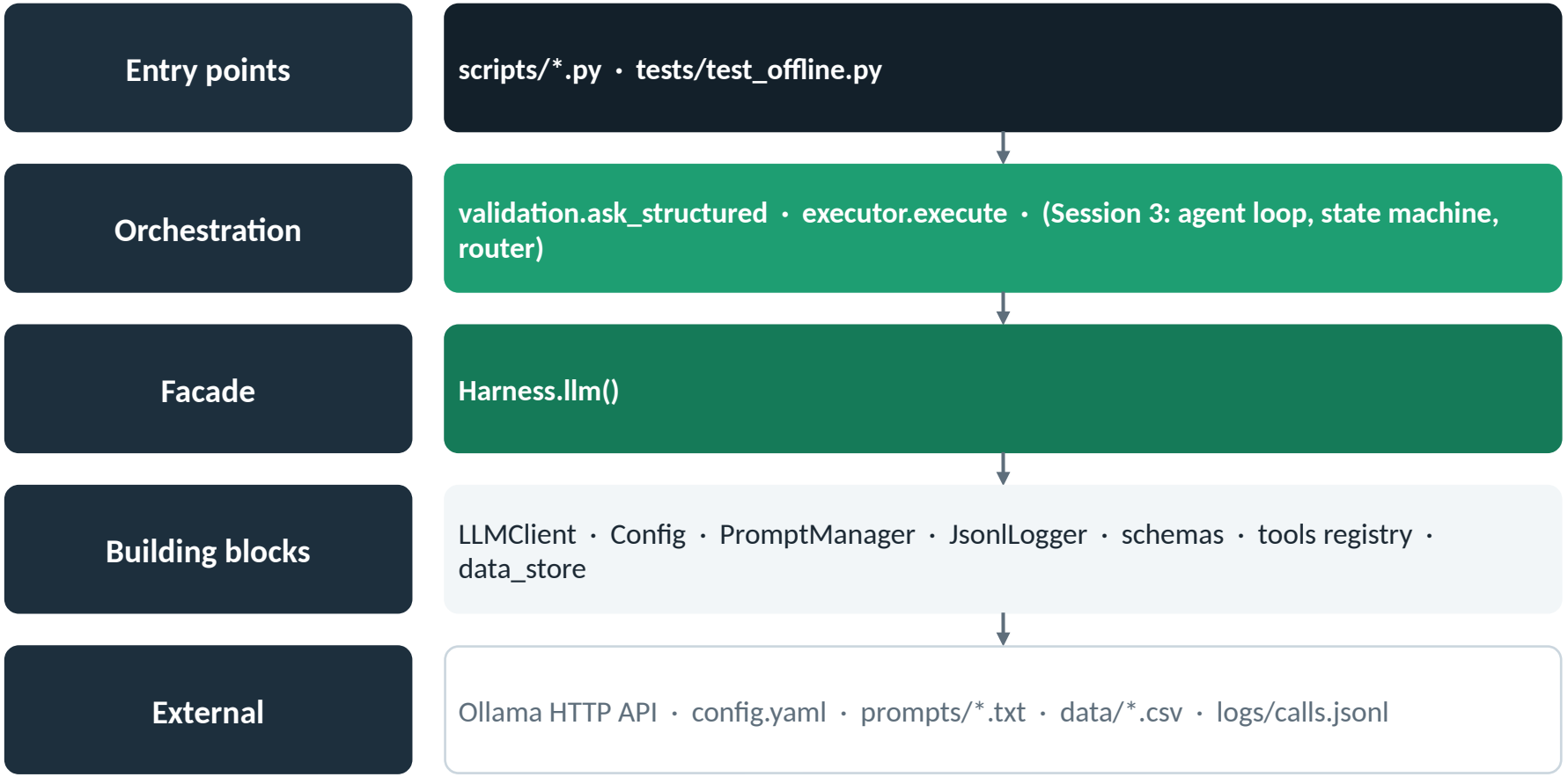
COMPONENTS

What each component owns

File	Main API	Responsibility	Depends on
llm_client.py (M1)	LLMClient.chat() → ChatResult	HTTP to Ollama; normalises native/OpenAI replies; maps errors	requests
config.py (M2)	load_config() → Config.profile()	Reads config.yaml into named Profiles	yaml
prompts.py (M3)	PromptManager.get(), build_messages()	Versioned templates; stable → variable order	—
harness.py	Harness.llm(profile, …)	Facade: resolve profile, call client, log	M1, M2, M3, M14
schemas.py (M4)	Ticket, schema_of()	Output shape → JSON schema for format	pydantic
validation.py (M5)	ask_structured(), check_ticket()	Schema + domain check; retry, repair, fallback, safe failure	Harness, M4, data_store
tools.py (M6)	@tool, TOOLS, tool_defs()	Registry of typed, permissioned tools	pydantic
executor.py (M7)	execute(), tool_message()	Look up, validate, permission, timeout; ok/err	M6, logger (optional)
sample_tools.py	calculator, lookup_customer, …	Example tools, registered at import	M6, data_store
logger.py (M14)	JsonlLogger.llm_call(), tool_call()	One JSONL line per event; run_id groups a task	—

RELATIONS

Dependencies point one way: downward



Why it matters

- Swap the runtime by changing only LLMClient (or config.yaml)
- Test orchestration offline with a FakeClient
- Add a layer on top without touching the blocks
- Nothing low-level imports the scripts

FACADE

Inside Harness.llm(): profile → request

config.yaml

```
profiles:
  fast:
    model: qwen3:4b-q4_K_M
    temperature: 0
    num_ctx: 8192
    num_predict: 512
    keep_alive: 10m
    think: false
```

h.llm("fast", msgs, schema=S, num_predict=256)

```
p = config.profile("fast")
options = {**p.options,
           **overrides}
client.chat(p.model, msgs,
   schema=S, think=p.think,
   keep_alive=p.keep_alive,
   **options)
```

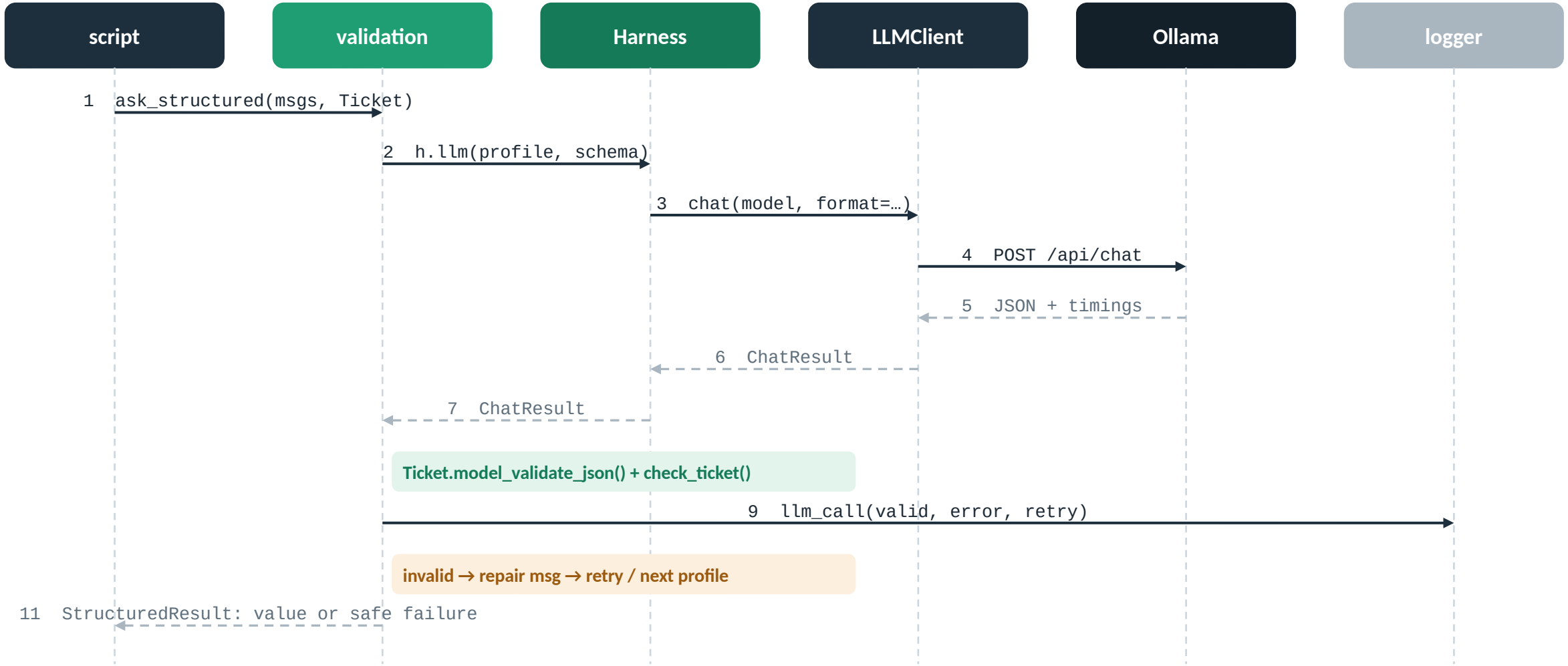
POST /api/chat

```
{"model": "qwen3:4b-q4_K_M",
 "messages": [...],
 "format": {JSON schema},
 "think": false,
 "keep_alive": "10m",
 "options": {"temperature": 0,
   "num_ctx": 8192,
   "num_predict": 256}}
```

- **Overrides win:** per-call keywords replace profile values for that call only.
- **Then log:** one llm_call event (tokens, TTFT, gen time, wall time), unless log=False.

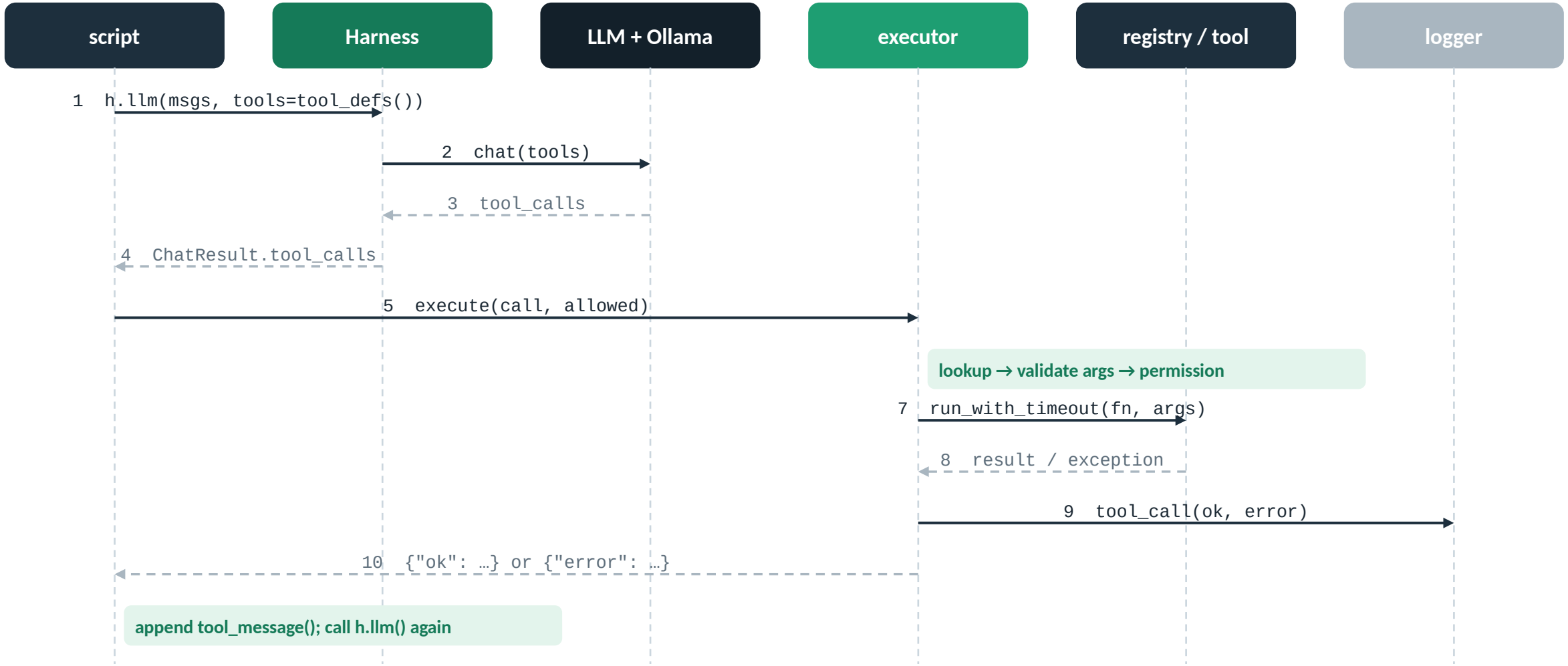
CALL FLOW A

Structured output: ask_structured()



CALL FLOW B

Tool call round trip



DATA CONTRACTS

What passes between components

ChatResult · from M1

model, content, message, tool_calls,
prompt_tokens, output_tokens,
load_s, prompt_s, gen_s, wall_s
+ ttft_s, gen_tps (computed)

Profile · M2 to facade

name, model,
options {temperature, num_ctx, num_predict, seed},
think, keep_alive

StructuredResult · from M5

ok, value, attempts,
first_try_valid, profile_used,
errors[], total_wall_s, reason

Tool call · model to M7

{"function": {"name": "...",
"arguments": {...} or "json"}}

Observation · M7 to model

{"ok": true, "result": ...}
{"ok": false, "error": "..."}
wrapped as {"role": "tool", ...}

Log event · all to M14

ts, event, run_id + fields
llm_call · tool_call · safe_failure

EXTENDING

Where to change what

I want to…	Edit	Nothing else changes because…
Use a different model	config.yaml (profile model)	code asks for profiles, not tags
Switch to vLLM / an OpenAI-style server	config.yaml: api: openai, base_url	LLMClient normalises both replies to ChatResult
Improve a prompt	add prompts/name@v3.txt	get("name") picks the latest; the log shows the version
Extract a new kind of object	new Pydantic model in schemas.py + a rule	ask_structured() is generic over the model class
Add a business rule	a check function raising DomainError	passed as domain_check=…
Add a tool	@tool function in sample_tools.py (or a new module)	registry and executor pick it up
Allow write tools	execute(call, allowed={"read", "write"})	permission is checked per call

NEXT

Where Session 3 plugs in

M8 Agent loop

Repeats flow B: h.llm(tools) → execute() → tool_message() until there are no tool calls or max steps is reached.

M9 State machine

Wraps the loop. Each state picks its prompt, schema, tool_defs(names) and allowed permissions.

M10–M11 Classifier + router

Choose the profile name passed to h.llm(). Fallback = the profiles tuple in ask_structured().

All three sit in the orchestration layer and reuse the same facade, validation and executor.